

PRG1 – Ensembles et tableaux associatifs

Méthodes de la classe `Iterator<T>`

boolean hasNext()	Retourne vrai si l'itération a d'autres éléments non parcourus.
T next()	Avance l'élément courant de l'itérateur et retourne ce nouvel élément. pré-requis : hasNext()
void remove()	Retire le dernier élément retourné par l'itérateur de la collection concernée. Ne peut pas être utilisé plus d'une fois par appel à <i>next</i> .

Constructeur de la classe `Set<T>`

Le constructeur vide crée un ensemble vide.

Méthodes de la classe `Set<T>`

boolean add(T element)	Rajoute <i>element</i> à l'ensemble, sans effet si déjà présent.
void clear()	Supprime tous les éléments de l'ensemble.
Set<T> clone()	Retourne une copie de l'ensemble.
boolean contains(T element)	Retourne vrai si <i>element</i> appartient à l'ensemble, faux sinon.
boolean isEmpty()	Retourne vrai si l'ensemble est vide, faux sinon.
Iterator<T> iterator()	Retourne un itérateur sur les éléments de l'ensemble. Positionne l'itérateur avant le premier élément.
boolean remove(T element)	Retire <i>element</i> de l'ensemble, sans effet si n'est pas présent.
int size()	Retourne le nombre d'éléments de l'ensemble.
T[] toArray()	Retourne un tableau contenant les éléments de l'ensemble.
boolean equals (Set<T> set)	Retourne vrai si l'ensemble est égal à <i>set</i> , faux sinon.
boolean included (Set<T> set)	Retourne vrai si l'ensemble est inclus dans <i>set</i> , faux sinon. pré-requis : !set.isEmpty()
boolean disjoint (Set<T> set)	Retourne vrai si l'ensemble est disjoint de <i>set</i> , faux sinon. pré-requis : !set.isEmpty()
void union (Set<T> set)	Rajoute à l'ensemble les éléments de <i>set</i> . pré-requis : !set.isEmpty()

void intersection (Set<T> set)	Retire de l'ensemble les éléments non présents dans <i>set</i> . pré-requis : !set.isEmpty()
void difference (Set<T> set)	Retire de l'ensemble les éléments présents dans <i>set</i> . pré-requis : !set.isEmpty()
void symmetricDifference (Set<T> set)	Réalise, dans l'ensemble, l'union moins l'intersection avec <i>set</i> . pré-requis : !set.isEmpty()

Constructeur de la classe Table<K, V>

Le constructeur vide crée un tableau associatif vide.

Méthodes de la classe Table<K, V>

void clear()	Supprime toutes les entrées de la table.
boolean contains(K key)	Retourne vrai si la table contient la clé <i>key</i> .
V getValue(K key)	Retourne la valeur associée à la clé <i>key</i> . Retourne null si la table ne contient pas la clé <i>key</i> .
boolean isEmpty()	Retourne vrai si la table est vide, faux sinon.
Iterator<K> iterator()	Retourne un itérateur sur les clés de la table. Positionne l'itérateur avant le premier élément.
void addValue(K key, V value)	Ajoute le couple (<i>key</i> , <i>value</i>) à la table, si <i>key</i> n'est pas une entrée de la table. Sans effet si <i>key</i> est une entrée de la table.
V modifyValue(K key, V value)	La nouvelle valeur associée à <i>key</i> devient <i>value</i> , si <i>key</i> est une entrée de la table. Sans effet si <i>key</i> n'est pas une entrée. Retourne l'ancienne valeur associée à la clé <i>key</i> .
V removeValue(K key)	Retire la clé <i>key</i> (et la valeur associée) de la table, si la clé est présente. Retourne l'ancienne valeur associée à la clé <i>key</i> .
int size()	Retourne le nombre d'entrées de la table.

On désire manipuler un réseau social type Twitter défini sur le type *int*; chaque réseau social *net* est représenté par une table à clés de type *int* et à valeurs de type *Set<Integer>* : à toute entrée *x* de *net* est associé l'ensemble (non vide) des entiers *y* tels que $(x, y) \in net$ (*x* est follower de *y*).

```
public class SocialNetwork extends Table <Integer, Set <Integer>>
{
    public SocialNetwork() { super(); }
    ...
}
```

QUESTIONS - Écrire les méthodes d'instance suivantes :

1.

```
/**
 * @return true si la relation (x,y) appartient à this, false sinon
 */
public boolean isDefined (Integer x, Integer y)
```
2.

```
/**
 * @return nombre d'entiers y tels que (x, y) appartient à this.
 */
public int numberOfYs (Integer x)
```
3.

```
/**
 * @return nombre de doublets (x,y) dans la relation this
 */
public int numberOfPairs ()
```
4.

```
/**
 * Ajouter la relation (x,y) à this (sans effet si (x, y) est
 * déjà présent)
 */
public void addRelation (Integer x, Integer y)
```
5.

```
/**
 * Supprimer la relation (x,y) de this (sans effet si (x, y) n'est
 * pas présent).
 */
public void removeRelation (Integer x, Integer y)
```
6.

```
/**
 * @return nombre de triplets (x, y, z) tels que (x,y) appartient
 * à this et (y,z) appartient à net
 */
public int join (SocialNetwork net)
```

7.

```
/**
 * @return relation symétrique de this, la relation constituée des
 * (y, x) tels que (x, y) appartient à this.
 */
public SocialNetwork symmetricRelation ()
```
8.

```
/**
 * @return true si this est une relation réflexive (i.e.~si pour
 * toute entrée x de this, (x, x) appartient à this), false sinon
 */
public boolean isReflexive ()
```
9.

```
/**
 * @return true si this est inclus dans net, false sinon
 */
public boolean isIncludedIn (SocialNetwork net)
```
10.

```
/**
 * @return intersection de this et net
 */
public SocialNetwork intersection (SocialNetwork net)
```
11.

```
/**
 * this devient l'intersection de this et net.
 */
public void intersectionBis (SocialNetwork net)
```
12.

```
/**
 * this devient l'union de this et net.
 */
public void union (SocialNetwork net)
```

Classe ArrayList

- La classe **ArrayList** est accessible par **import** `java.util.ArrayList`.
- Un objet `ArrayList <T>` est une **liste d'éléments du type T**, gérée en tableau, dont la taille peut augmenter dynamiquement (le tableau est recréé si nécessaire).
- L'accès à un élément peut s'effectuer à l'aide de la position, **entre 0 et taille-1**, dans la liste.
- On dispose, entre autres, des méthodes suivantes :
 - **constructeur ArrayList ()** : construit une liste vide ;
 - **int size ()** : délivre le nombre d'éléments de la liste ;
 - **boolean isEmpty ()** : délivre **vrai** si la taille vaut 0 (i.e. `size () == 0`) ;
 - **T get (int index)** : en entrée $0 \leq index < size()$, délivre la référence de l'élément situé à la position `index` ;
 - **void add (int index, T element)** : en entrée $0 \leq index \leq size()$, insère l'élément à la position spécifiée ; les éléments de position $\geq index$ avant l'ajout voient leur position augmentée de 1 ;
 - **T remove (int index)** : en entrée $0 \leq index < size()$, retire l'élément situé à la position `index` ; les éléments de position $> index$ avant le retrait sont décalés d'une position à gauche ; retourne l'élément qui vient d'être retiré ;
 - **void clear ()** : retire tous les éléments de la liste.

Classe Set, en représentation ordonnée, à l'aide de ArrayList

- Pour Set, il faut assurer la gestion des **doublons** et l'**ordre** sur les éléments ;
- De plus, afin de gérer efficacement les déplacements dans la liste selon l'ordre, on utilise l'**attribut entier position** :
 - lorsque $0 \leq position < size()$, `position` indique le rang de l'élément courant selon l'énumération induite par l'ordre ;
 - lorsque l'élément courant n'est plus défini (liste vide, recherche d'un élément supérieur au dernier, retrait du dernier élément, ...), on a `position == size ()`.

Compléter les méthodes add et remove de la classe Set

```
1 import java.util.ArrayList;
2 public class Set <T extends Comparable<T>> {
3     private ArrayList<T> list;
4     private int position; // position courante
5     public Set () {
6         list = new ArrayList<T>();
7         position = 0;
8     }
9     public boolean contains(T x) {
10    /* on repositionne position à 0 seulement si x figure dans la partie
11    déjà parcourue de liste, c'est-à-dire entre les indices 0 et
12    position-1, donc si x < list[position]. Ceci assure en particu-
13    lier que, pour la méthode intersection de l'exercice 3, l'appel
14    F.contains(x) ne recommence pas, lors de chaque itération, un
15    nouveau parcours de F et ce, même si F a été entièrement exploré.
16    */
17        if (list.size() == 0) { return false; }
18        if (position == list.size()) { --position; }
19        if (x.compareTo(list.get(position)) < 0) { position = 0; }
20        while (position < list.size() &&
21               x.compareTo(list.get(position)) > 0)
22            { ++position; }
23        return position < list.size() && x.equals(list.get(position));
24    }
25    public int size() { return list.size(); }
26    public boolean isEmpty() { return list.isEmpty(); }
27    public boolean add(T x) {
28
29
30
31
32    }
33    public boolean remove(T x) {
34
35
36
37
38    }
39    public void clear() { list.clear(); position = 0; }
40    public Iterator<T> iterator() {
41        return new Iterator<T>(list);
42    }
43 }
```

```
1 import java.util.ArrayList;
2 public class Iterator<T>{
3     private int current;
4     private ArrayList<T> list;
5
6     protected Iterator(ArrayList<T> l) {
7         current = -1;
8         list = l;
9     }
10
11     public boolean hasNext()
12     {
13         return current < list.size() - 1;
14     }
15
16     public T next()
17     {
18         assert this.hasNext() : "il a plus de nouvelles valeurs";
19         ++current;
20         return list.get(current);
21     }
22
23     public void remove()
24     {
25         list.remove(current);
26         --current;
27     }
28 }
```

Classe **HashMap**

- La classe **HashMap** de Java est accessible par `import java.util.HashMap`.
- Un objet **HashMap**<K, V> implémente, à l'aide du hachage, une table où les clés sont de type K et les valeurs associées sont de type V.
- La classe **HashMap** <K, V> fournit, entre autres, les méthodes suivantes :
 - constructeur **HashMap** () construit une table vide
 - int size () délivre le nombre de clés de la table
 - boolean isEmpty () délivre vrai si la table est vide (ie. size () == 0)
 - V get (K c) délivre la valeur associée à la clé c si c est une entrée de la table, **null** sinon
 - V put (K c, V v) ajoute l'entrée c de valeur v ; retourne **null** si c n'était pas une entrée ou bien la valeur avant son remplacement par v
 - V remove (K c) retire la clé c et sa valeur associée si c est une entrée ; retourne la valeur associée à c avant le retrait ou **null** si c n'était pas une entrée
 - void clear () retire toutes les entrées de la table
- La classe K doit comporter les méthodes :
 - `public int hashCode () { /* fonction de dispersion */ }`
car les méthodes nécessitant la recherche d'une clé c (get, put et remove) sélectionnent la sous-table susceptible de contenir c à l'aide de c.hashCode() ; si on ne fournit pas une méthode hashCode alors Java utilise une méthode par défaut qui calcule un entier à l'aide de la référence de l'objet et non à partir du contenu (donc si deux objets, de références différentes, repèrent des clés identiques leurs hashCode seront probablement différents et la recherche ne sera pas effectuée dans la bonne sous-table).
 - `public boolean equals (Object obj) { return egal ((K) obj); }`
car la recherche d'une clé dans une sous-table nécessite une comparaison des contenus et non des références (ce que fait la méthode equals fournie par défaut).
Lorsque K $\equiv$ String (ce qui est très souvent le cas) il n'y a rien à faire car la classe String contient déjà ces deux méthodes.

- Pour construire une suite des clés de T type `Iterator <K>`, la classe **HashMap** `<K, V>` offre la méthode :

`Set keySet ()` : délivre l'ensemble des clés de la table, l'interface `Set` spécifie une méthode `iterator ()` permettant de "séquentialiser" cet ensemble dans un `Iterator<K>`.

L'interface `Iterator <K>` spécifie les trois méthodes :

- `boolean hasNext ()`
- `K next ()`
- `void remove ()`

et on peut alors écrire :

```
1  Iterator <K> it = T.iterator ();
2      // l'objet itérateur est initialisé avec la suite
3      // des clés de T
4  it.hasNext ();
5      // délivre vrai si l'itération n'est pas terminée, faux sinon
6  it.next ();
7      // délivre la clé suivante de l'itérateur
8  it.remove ();
9      // supprime la dernière clé retournée par it.next()
10     // ainsi que la valeur associée à cette clé
```

Attention : il n'est pas possible d'utiliser les méthodes `put` et `remove` de la classe `HashTable` pendant le parcours. Cela engendre une exception (`java.util.ConcurrentModificationException`).

Classe `Iterator<T>`

```
1  import java.util.Iterator;
2  import java.util.Set;
3  public class Iterator<T> extends java.util.Iterator<T>{
4      protected Iterator(Set<T> e) {
5          it = e.iterator();
6      }
7  }
```

Classe Table <K, V>

```
1 import java.util.HashMap;
2
3 public class Table<K extends Comparable<K>, V> extends HashMap<K, V> {
4
5     public Table() {
6         super();
7     }
8
9     public V getValue(K key) {
10         V value = super.get(key);
11         return value;
12     }
13
14     public int size() {
15         return super.size();
16     }
17
18     public boolean isEmpty() {
19         return super.isEmpty();
20     }
21
22     public void clear() {
23         super.clear();
24     }
25
26     public void addValue(K key, V value) {
27         if (this.getValue(key) == null)
28             super.put(key, value);
29     }
30
31     public void modifyValue(K key, V value) {
32         if (this.getValue(key) != null)
33             super.put(key, value);
34     }
35
36     public V removeValue(K key) {
37         return super.remove(key);
38     }
39
40     public Iterator<K> iterator() {
41         return new Iterator<K>(this.keySet());
42     }
43 }
```